

Arquitetura de Computadores II

Execução e commit

REALIZAR APÓS posição 30: Recuperando de falhas e liberando recursos no estágio de commit

Posições na playlist 19 a 30 | 120 minutos | 100 pontos

Curso principal: MO601 2020s2 - Arquitetura de Computadores II |
https://www.youtube.com/playlist?list=PLEUHFTHcrJmsqKX-GDD-hBvkF8h2_BfKJ

Conteúdo verificável: Dependências; renomeação; issue; memória; unidades; bypass; clustering; commit; recuperação

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Dependências e renomeação

I1:R1←R2+R3; I2:R4←R1+R5; I3:R1←R6+R7. (a) Identifique RAW, WAR, WAW. (b) Renomeie os dois destinos R1 em registradores físicos distintos. (c) Explique qual dependência a renomeação não elimina.

Desenvolvimento / justificativa / esquema:

Issue e unidades

I1 produz um operando de I2; I3 é independente. (a) Explique quando I3 pode executar antes de I2 numa máquina fora de ordem. (b) Discuta unidades funcionais, leitura de operandos e rede de bypass. (c) Explique tradeoff de clustering.

Desenvolvimento / justificativa / esquema:

Operações de memória

Uma store antiga tem endereço ainda desconhecido; uma load jovem está pronta. (a) Analise risco de executar load cedo. (b) Proponha política conservadora e uma especulativa com recuperação. (c) Explique por que renomeação de registradores não resolve alias em memória.

Desenvolvimento / justificativa / esquema:

Commit e exceção precisa

I1 demora, I2 termina cedo e I3 provoca exceção. (a) Explique commit em ordem num processador que executa fora de ordem. (b) Descreva estado arquitetural ao tratar I3. (c) Discuta descarte de jovens e liberação de recursos.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Dependências e renomeação (15 pontos)

RAW:I1→I2 emR1; WAR:I2→I3 emR1; WAW:I1→I3 emR1. DarI1 destinoP1, I2 lêP1, eI3 destinoP2 remove conflitos de nomeWAR/WAW. RAW permanece porqueI2 precisa do resultado realI1. Tabela de mapa deve ser atualizada na ordem apropriada de allocation; renomeação não altera a semântica dos dados.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Issue e unidades (25 pontos)

Se fontesI3 prontas e há unidade livre, pode ser despachada enquantoI2 esperaI1. Bypass encaminha resultado antes de escrita/leitura posterior de registrador, quando suportado. Issue considera prontidão e recursos, não somente ordem textual. Clustering pode reduzir custo/atraso de interconexão, mas comunicação entre grupos e distribuição de operações podem criar penalidades. Preservar dependências verdadeiras e confirmação correta é obrigatório.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Operações de memória (25 pontos)

Se endereços coincidirem, load deve observar store anterior conforme modelo adotado. Conservadora espera determinar dependências; especulativa executa e verifica depois, refazendo/descartando se conflito. Endereços são valores computados e podem coincidir dinamicamente; nomes de registrador físicos distintos não provam independência em memória. A política deve garantir comportamento arquitetural ao commit.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Commit e exceção precisa (35 pontos)

Resultados executados podem ficar em estruturas especulativas; commit avança na ordem do programa quando cada instrução antiga está concluída e segura. Para exceção precisa em I3, I1/I2 anteriores devem refletir execução correta, e I3 e posteriores não devem deixar efeitos comprometidos indevidos. Descartar jovens/restaurar mapa e buscar no local correto, liberando entradas/registadores conforme política. Terminar execução não equivale a modificar irreversivelmente estado arquitetural.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.